

Building Neural Networks in Python

Yale Quan, Ph.D.

University of Washington

April 24, 2026

Workshop Agenda

- **Part 1: The Theory (30 mins)**
 - How Neural Networks learn
 - The Keras API
 - The 5-Step Deep Learning Workflow
- **Part 2: Hands-On Coding + Q&A (60 mins)**
 - Environment setup
 - Data preparation & shapes
 - Building, training, and tuning a model

What is a Neural Network?

- A machine learning algorithm inspired by the human brain.
- Designed to recognize patterns in data (numbers, text, images).
- Instead of hard-coding rules (e.g., if $X > 5$ then $Y = 1$), we provide examples, and the network "learns" the rules.

What is a Neural Network?

- A machine learning algorithm inspired by the human brain.
- Designed to recognize patterns in data (numbers, text, images).
- Instead of hard-coding rules (e.g., if $X > 5$ then $Y = 1$), we provide examples, and the network "learns" the rules.
- **Training Data (X / Features):** The information we know beforehand.
Example: Hours Studied, Previous Score, Attendance.
- **Training Labels (y / Targets):** The final answer we want the model to learn how to predict.
Example: Passed (1) or Failed (0).

Network Architecture: Deep, Shallow, Wide, and Narrow

We often hear these terms used to describe a model's **architecture** sometimes referred to as **topology** in more theoretical applications.

Network Architecture: Deep, Shallow, Wide, and Narrow

We often hear these terms used to describe a model's **architecture** sometimes referred to as **topology** in more theoretical applications.

Depth (Number of Layers)

- **Shallow:** 1 to 2 hidden layers. Best for simpler data. Faster to train and less likely to overfit.
- **Deep:** 3+ hidden layers (This is “Deep Learning”). Best for hierarchical data where patterns build on each other (e.g., combining edges into shapes, and shapes into faces).

Network Architecture: Deep, Shallow, Wide, and Narrow

We often hear these terms used to describe a model's **architecture** sometimes referred to as **topology** in more theoretical applications.

Depth (Number of Layers)

- **Shallow:** 1 to 2 hidden layers. Best for simpler data. Faster to train and less likely to overfit.
- **Deep:** 3+ hidden layers (This is “Deep Learning”). Best for hierarchical data where patterns build on each other (e.g., combining edges into shapes, and shapes into faces).

Width (Nodes per Layer)

- **Narrow:** Few nodes (e.g., 8, 16). Forces the network to compress information and only learn the most essential, defining features.
- **Wide:** Many nodes (e.g., 128, 512). Gives the model a massive capacity to capture diverse, complex patterns, but demands much more data to train effectively.

Network Architecture: Deep, Shallow, Wide, and Narrow

We often hear these terms used to describe a model's **architecture** sometimes referred to as **topology** in more theoretical applications.

Depth (Number of Layers)

- **Shallow:** 1 to 2 hidden layers. Best for simpler data. Faster to train and less likely to overfit.
- **Deep:** 3+ hidden layers (This is “Deep Learning”). Best for hierarchical data where patterns build on each other (e.g., combining edges into shapes, and shapes into faces).

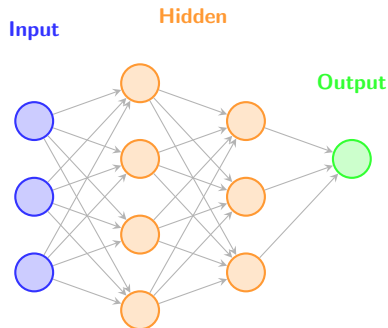
Width (Nodes per Layer)

- **Narrow:** Few nodes (e.g., 8, 16). Forces the network to compress information and only learn the most essential, defining features.
- **Wide:** Many nodes (e.g., 128, 512). Gives the model a massive capacity to capture diverse, complex patterns, but demands much more data to train effectively.

Rule of Thumb: *Start shallow and narrow. Only add depth and width when your model proves it isn't complex enough to learn the data!*

Anatomy of a Feedforward Network

- **Input Layer:** One node for each feature in your dataset.
- **Hidden Layers:** Where the mathematical transformations occur. "Deep" learning implies multiple hidden layers.
- **Output Layer:** The final prediction (e.g., a single probability for binary classification).



Data flows strictly from left to right during prediction.

How does the network actually learn?

- Every connection between nodes has a **Weight** (importance).
- Every node has a **Bias** (an offset).
- **The Goal of Training:** Find the exact combination of weights and biases that minimizes prediction error.
- It does this through an iterative process called **Gradient Descent**.

Weights & Biases

How does the network actually learn?

- Every connection between nodes has a **Weight** (importance).
- Every node has a **Bias** (an offset).
- **The Goal of Training:** Find the exact combination of weights and biases that minimizes prediction error.
- It does this through an iterative process called **Gradient Descent**.

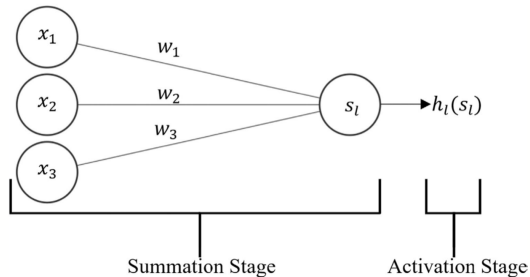


FIGURE A2 Neural network summation and activation stages.

$$s_l = \sum_{j=1}^J (x_j \times w_j) + b_l \quad (\text{A4})$$

Quan, Y., & Wang, C. (2026). Using multilabel classification neural network to detect intersectional DIF with small sample sizes. *British Journal of Mathematical and Statistical Psychology*, 00, 1–38. <https://doi.org/10.1111/bmsp.70041>

Adding Non-Linearity: Activation Functions

Without activation functions, a neural network is just a linear regression model.

- **ReLU (Rectified Linear Unit):**

- If input is negative, output 0. If positive, output the input.
- $\text{ReLU}(s_I) = \max(0, s_I)$
- Usually used for classification networks

- **Sigmoid:**

- Maps any real number into a value between 0 and 1.
- $\sigma(s_I) = \frac{1}{1+e^{-s_I}}$
- Usually used as an output activation function for binary classification

- **Linear:**

- No activation is applied
- $f(s_I) = s_I$
- Usually used for a prediction (or regression) network

NOTE: If you do not specify an activation function Keras assumes you want a Linear activation.

Classification vs. Regression in Keras

The strength of Keras is that changing the nature of your model often *sometimes* just requires changing the **Output Layer** and the **Loss Function**. You should always perform a check.

Binary Classification

- *Goal*: Predict a category (e.g., Pass/Fail).
- *Output Nodes*: 1
- *Activation*: sigmoid (Probabilities)
- *Loss*: `binary_crossentropy`

Regression

- *Goal*: Predict a continuous number (e.g., Final Score).
- *Output Nodes*: 1
- *Activation*: linear (or omit)
- *Loss*: `mse` (Mean Squared Error)

Why Keras?

- **High-Level API:** Designed for humans. It translates your intent into complex math.
- **TensorFlow Engine:** Runs on top of Google's TensorFlow for production-grade speed.
- **Lego Blocks:** Building a model is just stacking layers like blocks.

Why Keras?

- **High-Level API:** Designed for humans. It translates your intent into complex math.
- **TensorFlow Engine:** Runs on top of Google's TensorFlow for production-grade speed.
- **Lego Blocks:** Building a model is just stacking layers like blocks.

Drawbacks

- **Deceptively Simple:** Keras is easier to implement than others, but its error messages are hard to understand and you can easily specify wrong models that converge.
- **TensorFlow:** TensorFlow is very powerful but fragile. Methods are highly version dependent and updates often break existing workflows. (Tip: freeze the TensorFlow version your research requires)
- **Computation Power:** Tensorflow requires significant computation power. Especially when tuning a model (more later) or building models with very complex architecture.

The Keras Workflow

- 1 Prepare Data
- 2 Define Architecture
- 3 Compile Model
- 4 Fit Model
- 5 Evaluate & Predict

Step 1: Data Preparation

Neural networks are brittle. They require strict data formats:

- **Numbers Only:** All categorical text must be encoded.
- **Missing Data:** Most networks cannot handle missing data.
- **Scaling is Recommended:** Features should generally be scaled between 0 and 1, or standardized (mean 0, variance 1). Large unscaled numbers produce unstable gradients.
- **Train/Test Split:** We must hold out data the model has never seen to test if it actually learned, or just memorized.

Step 1.5: Understanding Shapes

The most common error beginners face in Keras is a *shape mismatch*.

- Data are passed into the network using batches. This defines the **shape** of the network.
- If you have 100 rows and 5 features, your data shape is (100, 5).
- The **Input Layer** must be explicitly told to expect 5 features.
- If you train a network using a data shape (100, 5) and then try to fit it to data with shape (100, 10) the model will crash and the error is not very helpful.

Example Error:

```
1 ValueError: Input 0 of layer "dense" is incompatible with the layer:  
   expected axis -1 of input shape to have value 5, but received input with  
   shape (None, 10)
```

Step 2: Defining the Architecture

```
1 from tensorflow.keras.models import Sequential
2 from tensorflow.keras.layers import Dense
```

Step 2: Defining the Architecture

```
1 from tensorflow.keras.models import Sequential
2 from tensorflow.keras.layers import Dense
3
4 # Initialize an empty sequential model
5 model = Sequential()
```

Step 2: Defining the Architecture

```
1 from tensorflow.keras.models import Sequential
2 from tensorflow.keras.layers import Dense
3
4 # Initialize an empty sequential model
5 model = Sequential()
6
7 # Add a hidden layer (must specify input_shape on the first layer)
8 model.add(Dense(64, activation='relu', input_shape=(5,)))
9
10 # Add another hidden layer
11 model.add(Dense(32, activation='relu'))
```

Step 2: Defining the Architecture

```
1 from tensorflow.keras.models import Sequential
2 from tensorflow.keras.layers import Dense
3
4 # Initialize an empty sequential model
5 model = Sequential()
6
7 # Add a hidden layer (must specify input_shape on the first layer)
8 model.add(Dense(64, activation='relu', input_shape=(5,)))
9
10 # Add another hidden layer
11 model.add(Dense(32, activation='relu'))
12
13 # Output layer: 1 node, sigmoid for binary probability
14 model.add(Dense(1, activation='sigmoid'))
```

Step 3: Compiling the Model

We must define the rules for training before we begin.

```
1 model.compile(  
2     optimizer='adam',           # The algorithm updating the weights  
3     loss='binary_crossentropy', # How we calculate error  
4     metrics=['accuracy']       # Human-readable performance tracking  
5 )  
6
```

Note: `adam` is a default optimizer that dynamically adjusts its learning rate.

The Compile Step: Loss vs. Metrics

Loss is the mathematical penalty the optimizer uses to update weights.

Metrics are human-readable scores to help you monitor performance.

The Compile Step: Loss vs. Metrics

Loss is the mathematical penalty the optimizer uses to update weights.

Metrics are human-readable scores to help you monitor performance.

Classification (Categories)

- **Loss:** `binary_crossentropy`
 - For Yes/No, Pass/Fail outcomes.
- **Loss:** `categorical_crossentropy`
 - For 3+ categories (e.g., predicting Letter Grades A, B, C, D, F).
- **Metric:** `accuracy`
 - The simple percentage of correct guesses.

The Compile Step: Loss vs. Metrics

Loss is the mathematical penalty the optimizer uses to update weights.

Metrics are human-readable scores to help you monitor performance.

Classification (Categories)

- **Loss:** `binary_crossentropy`
 - For Yes/No, Pass/Fail outcomes.
- **Loss:** `categorical_crossentropy`
 - For 3+ categories (e.g., predicting Letter Grades A, B, C, D, F).
- **Metric:** `accuracy`
 - The simple percentage of correct guesses.

Regression (Continuous Numbers)

- **Loss:** `mse` (Mean Squared Error)
 - Squares the errors, which heavily penalizes massive outliers. Good for gradient descent.
- **Metric:** `mae` (Mean Absolute Error)
 - Highly interpretable: *"On average, our prediction is off by X points on the final exam."*

The Compile Step: Loss vs. Metrics

Loss is the mathematical penalty the optimizer uses to update weights.

Metrics are human-readable scores to help you monitor performance.

Classification (Categories)

- **Loss:** `binary_crossentropy`
 - For Yes/No, Pass/Fail outcomes.
- **Loss:** `categorical_crossentropy`
 - For 3+ categories (e.g., predicting Letter Grades A, B, C, D, F).
- **Metric:** `accuracy`
 - The simple percentage of correct guesses.

Regression (Continuous Numbers)

- **Loss:** `mse` (Mean Squared Error)
 - Squares the errors, which heavily penalizes massive outliers. Good for gradient descent.
- **Metric:** `mae` (Mean Absolute Error)
 - Highly interpretable: *"On average, our prediction is off by X points on the final exam."*

Note: Always monitor the **validation** metrics (e.g., `val_loss`) to detect overfitting! Many other loss function and metrics exist. **You can also write your own!**

Step 4: Training (Fitting) the Model

```
1 history = model.fit(  
2     X_train,  
3     y_train,  
4     epochs=50,          # Full passes through the dataset  
5     batch_size=32,      # Rows processed before updating weights  
6     validation_split=0.2 # Automatically hold out 20% for testing  
7 )  
8
```

Step 4: Training (Fitting) the Model

```
1 history = model.fit(  
2     X_train,  
3     y_train,  
4     epochs=50,          # Full passes through the dataset  
5     batch_size=32,      # Rows processed before updating weights  
6     validation_split=0.2 # Automatically hold out 20% for testing  
7 )  
8
```

Note:

- ❶ As you increase the number of epoch the network *usually* becomes more accurate, but this will add computational complexity.
- ❷ Increase or decreasing the batch size can help the model learn more complex patterns.
- ❸ The default validation split is naive. It will just split the training data into chunks. Writing code to perform specific splitting rules is often better.

What happens during `model.fit()`?

Phase 1: Training (Learning the Data)

- ① **Forward Pass:** Training batches go in, predictions come out.
- ② **Loss Calculation:** Compare predictions to the actual answers.
- ③ **Backpropagation:** Calculate how much each weight contributed to the error.
- ④ **Update:** The Optimizer adjusts the weights to reduce the error.

What happens during `model.fit()`?

Phase 1: Training (Learning the Data)

- ➊ **Forward Pass:** Training batches go in, predictions come out.
 - ➋ **Loss Calculation:** Compare predictions to the actual answers.
 - ➌ **Backpropagation:** Calculate how much each weight contributed to the error.
 - ➍ **Update:** The Optimizer adjusts the weights to reduce the error.
-

Phase 2: Validation (The Reality Check)

- At the end of every epoch, the model evaluates the held-out **Validation Data**.
- The model is fit to the unseen data and the predicted values are compared to the ground truth labels.
- The model calculates loss and metrics, but **does not update its weights**.
- This acts as an independent check to ensure the model is learning general patterns, rather than just memorizing the training set.

Step 5: Evaluation & Prediction

```
1 # Evaluate on the final, unseen test set
2 loss, accuracy = model.evaluate(X_test, y_test)
3 print(f"Final Test Accuracy: {accuracy:.4f}")
4
5 # Generate predictions for brand new data
6 new_probabilities = model.predict(X_new_data)
7
8 # Convert probabilities to class labels (0 or 1)
9 new_classes = (new_probabilities > 0.5).astype("int32")
10
```

- `X_test` is the held out testing data observations.
- `y_test` is the ground truth for the testing data.
- `X_new_data` is the new data we collect without labels.

Monitoring Training: Overfitting

During the hands-on session, watch your `val_loss` (Validation Loss).

- If Training Loss goes down, but Validation Loss goes up...
- Your model is **Overfitting**. It is memorizing the training data and failing to generalize.
- *Solution*: Stop training earlier, use a simpler model (fewer nodes), or get more data. Keras has a helpful function called `EarlyStopping` and `callback` to help.

Beyond Feedforward: Specialized Architectures

So far, we have built **Feedforward (Dense)** networks, where data flows straight from input to output. For more advanced models, Keras has built-in layers for specialized data structures:

Beyond Feedforward: Specialized Architectures

So far, we have built **Feedforward (Dense)** networks, where data flows straight from input to output. For more advanced models, Keras has built-in layers for specialized data structures:

- **Convolutional Neural Networks (CNNs)**

- *The Concept:* They use sliding mathematical filters to detect spatial patterns like edges, shapes, and textures.
- *Best for:* Grid-like data such as images, computer vision tasks, or scanning documents.
- *Keras Layer:* Conv2D

Beyond Feedforward: Specialized Architectures

So far, we have built **Feedforward (Dense)** networks, where data flows straight from input to output. For more advanced models, Keras has built-in layers for specialized data structures:

- **Convolutional Neural Networks (CNNs)**

- *The Concept:* They use sliding mathematical filters to detect spatial patterns like edges, shapes, and textures.
- *Best for:* Grid-like data such as images, computer vision tasks, or scanning documents.
- *Keras Layer:* Conv2D

- **Recurrent Neural Networks (RNNs & LSTMs)**

- *The Concept:* They process data one step at a time, maintaining an internal state or "memory" of previous inputs to understand context.
- *Best for:* Sequential data like time-series tracking, natural language processing, or chronological sequences of student test responses.
- *Keras Layer:* LSTM or SimpleRNN

Callbacks (Smart Training)

Callbacks are functions that Keras runs at the end of every epoch to monitor training and take action automatically.

- **EarlyStopping:** Halts training the moment validation loss stops improving. You can specify the convergence criteria (Defaults `min_delta = 0` zero) and the value to monitor (Default: `'val_loss'`)
- **ModelCheckpoint:** Automatically saves the weights of your *best* epoch, rather than your *last* epoch.

```
1 from tensorflow.keras.callbacks import EarlyStopping
2
3 # Stop training if val_loss doesn't improve for 5 epochs in a row
4 early_stop = EarlyStopping(monitor='val_loss', patience=5,
5                             min_delta = 0.001)
6
7 history = model.fit(X_train, y_train, epochs=100,
8                     callbacks=[early_stop]) # Pass to the fit function
9
```

The Functional API

So far, we used the `Sequential` model (a straight line). But real-world research often requires complex, branching architectures (e.g., combining student test responses with demographic features to detect assessment bias).

The **Functional API** allows you to build models with multiple inputs, multiple outputs, or shared layers.

```
1 from tensorflow.keras.layers import Input, Dense,
   Concatenate
2 from tensorflow.keras.models import Model
3
4 # Two separate data streams
5 test_responses = Input(shape=(50,))
6 demographics = Input(shape=(5,))
7
8 # Process streams independently
9 branch_A = Dense(32, activation='relu')(test_responses)
10 branch_B = Dense(8, activation='relu')(demographics)
11
12 # Merge them together for a final prediction
13 merged = Concatenate()([branch_A, branch_B])
14 output = Dense(1, activation='sigmoid')(merged)
15
16 model = Model(inputs=[test_responses, demographics],
17               outputs=output)
```

- Layers are treated like functions:
`Layer()(previous_layer)`
- Essential for modern machine learning research.
- Highly flexible, yet uses the exact same `compile()` and `fit()` workflow!

Automating the Search with KerasTuner

Manually guessing the best number of nodes or layers is challenging. **KerasTuner** automates the trial-and-error process by training dozens of models and keeping the best one.

```
1 import keras_tuner as kt
2 from tensorflow.keras.layers import Input, Dense
3 from tensorflow.keras.models import Sequential
4
5 # 1. Define a model-building function with "hp" (hyperparameters)
6 def build_model(hp):
7     model = Sequential()
8     model.add(Input(shape=(3,)))
9
10    # Let the tuner choose the number of nodes (between 8 and 32)
11    hp_units = hp.Int('units', min_value=8, max_value=32, step=8)
12    model.add(Dense(hp_units, activation='relu'))
13
14    model.add(Dense(1, activation='sigmoid'))
15    model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy',
16    ])
17    return model
```

Automating the Search with KerasTuner

Manually guessing the best number of nodes or layers is challenging. **KerasTuner** automates the trial-and-error process by training dozens of models and keeping the best one.

```
1
2 # 2. Instantiate the Tuner (Random Search, Hyperband, etc.)
3 tuner = kt.RandomSearch(
4     build_model,
5     objective='val_accuracy',
6     max_trials=5 # Try 5 different combinations
7 )
8
9 # 3. Start the search!
10 tuner.search(X_train_scaled, y_train, epochs=10, validation_split=0.2)
11
12
```

Note: This is not always the best way to tune. This is very computationally expensive method. Sometimes you can review prior research to inform the tuning and you can specify a more fine-tuned grid.

Manual Tuning

We can also manually tune (which is usually what I do). You can create a search grid and write keras and python code that iterates over the grid and save any metrics you want. We can do this using JSON files

```
1
2 #Three Group Training JSON
3 {
4     "groups": 3,
5     "architectures": ["Single_Funnel", "Contextual"],
6     "split_depths": [0, 1, 2, 3],
7     "use_within_features": [true, false],
8     "trunk_configs": [[32, 16], [64, 32], [128, 64], [128, 64, 32], [64,
9 32, 16]],
10    "dropout_rates": [0.1, 0.2],
11    "learning_rates": [0.0001, 0.001],
12    "batch_sizes": [64, 128, 256],
13    "loss_weight_a": [1.0, 1.25, 1.5, 2.0]
14 }
```

Note: This results in 3,840 configurations to test. While this is very large, the grid is targeted for values I have identified as important. This takes about 1 month to run.

Time to open your notebooks.

We will now work through a complete end-to-end classification and regression example together.

Please navigate to the provided Colab link to get started.